# D4 — Deployment Demonstration and Dashboard

**Boréal Marché Customer Intelligence Platform** · Capstone Project 1

Olga Morgunova (individual project) · 420-977-VA AI Development Project · Instructor: Steve Ataky, Ph.D. · Vanier College, LEA.EU · Repository tag `v0.9` · KLO 4

## 1. Live system and evidence index

| Item | Where | Status |
|---|---|---|
| Application (5 sections, EN/FR toggle) | **PENDING** | App Runner service `boreal-ui` |
| API documentation (OpenAPI) | **PENDING** | App Runner service `boreal-api` |
| `/version` = deployed commit | **PENDING** | asserted by the CD smoke test (§5) |
| Screen recording, 3–5 min (prediction, recommendation, assistant) | **PENDING** | link also in `README.md` |
| CI: pull request with all checks green; workflow file | PR `feat/d4` → `master`; `.github/workflows/ci.yml` | §4 |
| CD: merge-triggered deploy, SHA-tagged image, smoke test; OIDC | Actions run of `.github/workflows/cd.yml` | §5 |
| Monitoring dashboard with shifted cohort and triggered alerts | *Monitoring* tab; evidence in §7 | measured locally on the release image |

Items marked PENDING are produced by the one-time AWS bootstrap and the first merge (`docs/runbooks/AWS_APPRUNNER_DEPLOY.md`); this note is regenerated from the evidence files once they exist (`python scripts/collect_d4_evidence.py --api-url … --ui-url … --video-url …`, then `python -m boreal_marche.reporting.build D4`). Nothing below is typed in by hand.

## 2. What is deployed

One API container serves all three heads and the monitoring endpoints; one interface container talks to it over HTTP only. Offline artifacts (model, rules, retrieval index, reference profile) are baked into the API image at build time, so a deployed version is exactly one image digest.

| Endpoint | Purpose |
|---|---|
| `/health`, `/ready` | liveness (process up) vs readiness (model, rules, index, reference loaded; 503 otherwise) |
| `/version` | API version, deployed commit, image tag, model version and source, rules mining date |
| `/predict`, `/predict/batch` | Head A — calibrated P(order within 90 days) and the *proposed* coupon decision |
| `/recommend`, `/products` | Head B — rules indexed by antecedent, strategy that fired; catalogue search |
| `/customers`, `/customers/{ref}/features` | pseudonymised demo customers: the feature-store stand-in |
| `/chat`, `/chat/stream`, `/chat/providers` | Head C — grounded assistant (JSON or Server-Sent Events) |
| `/metrics`, `/monitoring/report`, `/monitoring/alerts`, `/monitoring/timeseries` | Prometheus metrics; drift/quality/ops report with traffic lights; alert history |

Every request model is Pydantic v2 with `extra="forbid"`; errors are typed (422 validation, 404 unknown customer, 503 artifact or provider unavailable, 500 with a request id — never a bare stack trace). Models, rules and the index load once in the lifespan handler. Every request is written to `request_log` and as a JSON log line; every prediction (full feature vector), recommendation and chat turn to its own table — the dashboard is built from these rows. **Tests:** 80 pass, line coverage 84% (CI gate 70%), including the API contract suite and the golden-prediction regression test, which the post-deploy smoke test repeats against the live service.

**Head C, built for this deliverable.** 6 policy documents (4,995 words: delivery, returns, allergens, loyalty, privacy in EN+FR, ordering) plus the catalogue export, chunked by section into 66 chunks with metadata and embedded with `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (ONNX, no torch), so French questions retrieve English chunks. Four typed tools — `policy_search`, `retention_score` and `recommendations` (which call this API's own `/predict` and `/recommend` over HTTP) and `product_lookup` — plus an explicit `out_of_scope` choice. Guardrails run around every provider: another customer's data is refused before any model runs; `retention_score` has no customer argument at all; a policy answer below similarity 0.30 is handed to a human; every grounded answer carries citations; an LLM answer that mentions a SKU or percentage absent from the tool output is replaced by the verified template.

| Question set | Provider | Tool choice | Source cited | OOS refused | False refusals | Latency p50 | Cost/query |
|---|---|---|---|---|---|---|---|
| golden (40) | rule | 100.0% | 95.0% | 100.0% | 0.0% | 4 ms | $0.0000 |
| golden (40) | ollama | 97.5% | 90.0% | 100.0% | 3.2% | 1553 ms | $0.0000 |
| golden | openai | not run: OPENAI_API_KEY not set | | | | | |
| paraphrase (20) | rule | 35.0% | 22.2% | 80.0% | 73.3% | 0 ms | $0.0000 |
| paraphrase (20) | ollama | 95.0% | 88.9% | 100.0% | 6.7% | 1282 ms | $0.0000 |
| paraphrase | openai | not run: OPENAI_API_KEY not set | | | | | |

*Table 1 — Assistant evaluation (`reports/metrics/assistant_eval.json`). The golden set (40 questions) was written before the assistant; the paraphrase set (20) after the rule provider's keyword lists, to measure their overfitting. OpenAI is implemented behind the same interface but not evaluated: no API key (course §8.5 also forbids sending personal data to it).*

The rule baseline scores 100% on the golden set and 35% on paraphrases: its keywords were written by someone who had read the golden questions. The local model (Ollama, llama3.1:8b) holds up — 97.5% and 95% — at ~1.5 s per answer and $0. That is the measured case for the language model: routing natural phrasings and answering French questions from English sources. The rule-tool-choice check on the golden set runs in CI.

## 3. Containers

| Image | Unpacked size | Compressed (pushed to ECR) | Contents |
|---|---|---|---|
| API v0.3 (D3, single venv, `pip install .`) | 1437 MB | 719 MB | Heads A+B only |
| API v0.9 (this Dockerfile) | 823 MB | 407 MB | Heads A+B+C, monitoring, 241 MB embedding model baked in |
| Interface v0.9 | 500 MB | 155 MB | Streamlit + Altair, no model code |

*Table 2 — Image sizes measured on the built images (`reports/metrics/d4_evidence.json`).*

The API image is **43% smaller** than the D3 image while now also containing the assistant and its embedding model. Design choices in `docker/Dockerfile`:

| Choice | Why |
|---|---|
| Base `python:3.11-slim` **pinned by digest** | a tag can be re-pointed; a digest makes the OS layer reproducible |
| Multi-stage: `builder` → `runtime` | pip caches, build tools and wheel test suites stay in the builder |
| Serving lock `docker/requirements-api.txt` | only what the API imports; `xgboost-cpu` drops ~200 MB of CUDA/NCCL wheels; no pyarrow, MLflow, Optuna, matplotlib |
| Dependencies copied before source | a code change rebuilds one small layer, not 60 wheels |
| Embedding model downloaded at build time, `HF_HUB_OFFLINE=1` | no network at start-up; cold start does not depend on Hugging Face |
| Non-root user UID 10001 | a compromised process cannot write the image; CI asserts the UID |
| `HEALTHCHECK` on `/ready`, not `/health` | healthy only once every artifact is loaded |
| `GIT_SHA` / `IMAGE_TAG` as the last layer | `/version` proves which commit runs; changing it invalidates nothing else |
| Allow-list `.dockerignore` (deny `*`, allow 13 paths) | data, notebooks, MLflow state, `.env` can never reach the build context |

`docker compose -f docker/compose.yml up --build -d` starts API, interface, Postgres (prediction log) and Ollama with a one-shot model pull. I tested API + interface + Postgres from the images (7/7 smoke checks, predictions persisted in Postgres); the Ollama container's 4.9 GB model pull was not repeated in the test (the evaluation in Table 1 used the same model via a host install).

## 4. Continuous integration

`.github/workflows/ci.yml` runs on every pull request: **ruff** (lint + format), **mypy --strict**, **pytest with a 70% coverage gate** (unit, API contract, golden prediction, monitoring, assistant guardrails), the **data-quality suite**, the **assistant tool-choice evaluation** on the golden set, a **Docker build of both images with their sizes in the job summary**, a **container smoke test** (runs the API image, checks UID 10001, the health status and the same 6 API checks as CD), and **gitleaks** over the full history. Branch protection on `master` requires the three jobs and a pull request. Retrieval in CI uses a deterministic hashing embedder (`BOREAL_EMBEDDER=hash`) so tests need no model download; retrieval *quality* is measured by the evaluation in Table 1, not by CI.

## 5. Continuous deployment and OIDC

On merge to `master`, `.github/workflows/cd.yml`: (1) requests a GitHub OIDC token and exchanges it for one-hour credentials of role `BorealGithubDeploy` (`aws-actions/configure-aws-credentials`); (2) prints the assumed identity; (3) builds both images tagged with the **commit SHA** and pushes them to ECR (immutable tags, scan on push); (4) points the two App Runner services at the new tags (`scripts/apprunner_deploy.py`) and waits for success; (5) runs `scripts/smoke_test.py`, which fails the job unless `/version` equals the merged commit, the golden prediction is reproduced, a rule fires and the assistant cites a source.

**Why OIDC.** No AWS key exists in GitHub: the role's trust policy (`infra/aws/github-oidc-trust.json`) accepts only tokens whose subject is `repo:OlgaMorgunova@169389466/boreal-marche-capstone@1370564309:environment:production`, and the environment is restricted to `master`. Its permissions (`infra/aws/github-deploy-policy.json`) are push to two repositories, update/describe two services, pass one role to App Runner. A leaked log or fork cannot deploy, and nothing needs rotating. *Debugged in practice:* the first CD run was refused (`AccessDenied`) because I had written the subject as `repo:owner/repo:…`; CloudTrail's `AssumeRoleWithWebIdentity` event showed that GitHub's subject carries the immutable owner and repository IDs, so the trust policy now matches that form — which also means a recreated repository with the same name could not deploy.

**Scheduled retraining.** `.github/workflows/retrain.yml` (Mondays) re-runs ETL, retrains, re-mines rules, logs to MLflow, and evaluates the promotion policy against the committed champion on the validation snapshot — now including a **fairness gate** (subgroup lapser recall by region, tenure band and spend quartile; `boreal_marche.retention.promote`). Only if every gate passes does it open a pull request; it never deploys. A dry run with the extra-trees challenger correctly returned "no promotion". The same gate exposed a real gap that D5 must

address: lapsers in the top spend quartile receive the offer far less often than average (documented in `docs/MLFLOW_PROMOTION_POLICY.md`).

## 6. Hosting and cost

App Runner was chosen over Render because it deploys straight from ECR with the OIDC role and needs no second account. **Region deviation:** App Runner is not offered in `ca-central-1` (console message, checked 2026-10-24), so ECR and App Runner run in `us-east-1`, the closest supported region; the SageMaker job stays in `ca-central-1`. This is acceptable for the demonstration because the deployed service only processes the public, pseudonymised UCI data. For real customers it would be a transfer of personal information outside Québec (Law 25, s. 17: prior assessment, adequate protection), and I would deploy the same image to ECS Fargate behind an Application Load Balancer in `ca-central-1` (≈ $30/month more, mostly the load balancer) — the cost of data residency, stated rather than hidden. The API needs ~750 MiB at idle (measured with `docker stats`), so it runs at 1 vCPU / 2 GB; the interface at 0.25 vCPU / 0.5 GB. Estimated ≈ $13/month while running, $0 while paused; both services are paused between grading windows (runbook §4). Single instance by design: the SQLite prediction log lives on the instance and is lost on redeploy — the Compose stack shows the production design (Postgres). Ollama is not deployed (an 8B model needs a GPU instance); the live assistant offers the rule provider.

## 7. Monitoring with simulated and shifted traffic

The traffic simulator is a **Locust** file adapted from the course's Locust lab (`loadtest/locustfile.py`: `HttpUser`, weighted tasks, `catch_response`, grouped names). Phase 1 replays the 300 held-out demo customers (1821 requests, 0 failures, 15.3 req/s); phase 2 sends a deliberately shifted cohort — recent Maritimes sign-ups with one or two orders, no cadence history, lower spend, and baskets of SKUs the rules have never seen (1817 requests, 0 failures). Both phases ran against the release image; the report was saved after each (`scripts/simulate_traffic.py`).

| Check | After normal traffic | After shifted cohort | Measure |
|---|---|---|---|
| input drift | amber (0.000) | red (0.528) | share of features with PSI >= red |
| prediction drift | green (0.032) | red (2.112) | PSI of p_return |
| data quality | green (0.042) | red (0.994) | max(missing-rate change, /predict 422 rate) |
| error rate | green (0.000) | green (0.000) | share of 5xx responses |
| latency p95 predict | green (67.317) | green (70.200) | ms, server-side |
| rule fire rate | green (1.000) | red (0.188) | share of non-empty baskets answered by a rule |

*Table 3 — Traffic lights after each phase (`reports/metrics/monitoring_*.json`). Thresholds: `params.yaml → monitoring`, explained in `docs/MONITORING.md`.*

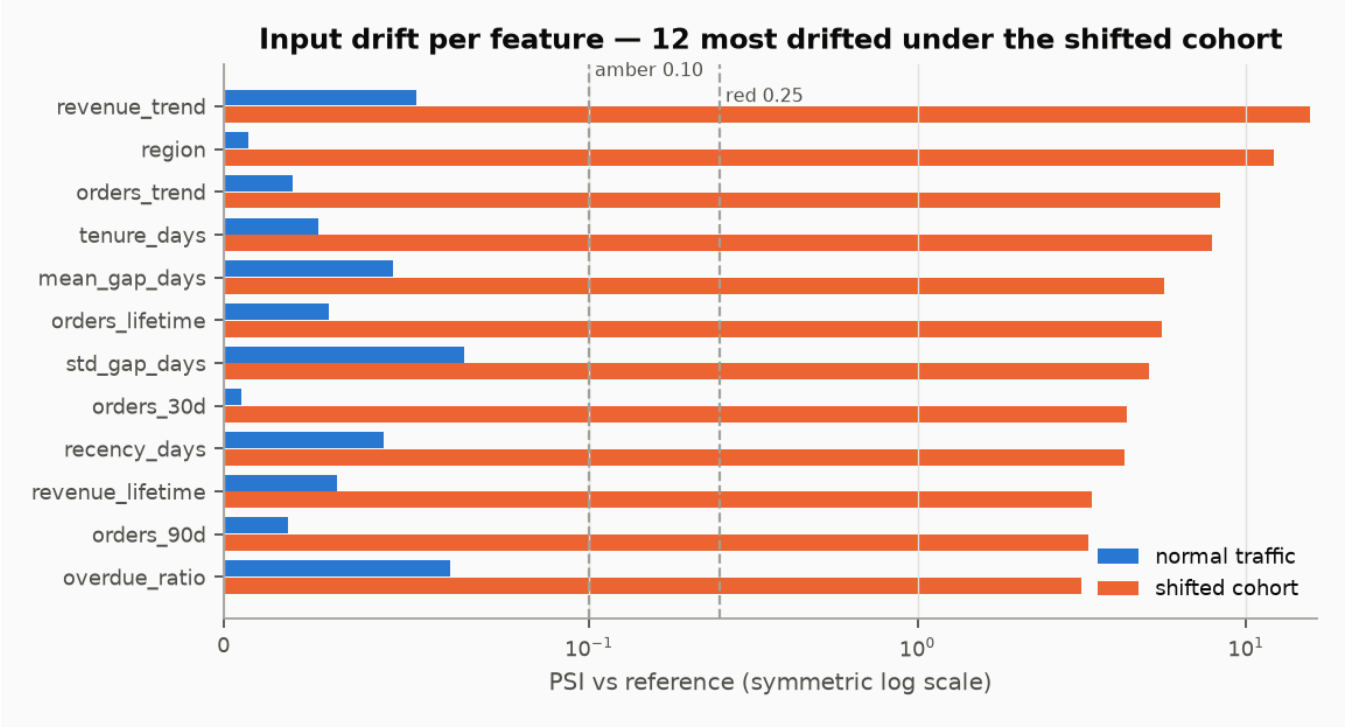



*Figure 1 — Input drift per feature, normal vs shifted traffic (top drifted: `revenue_trend` (PSI 15.63), `region` (PSI 12.12), `orders_trend` (PSI 8.35), `tenure_days` (PSI 7.84)).*

Under the shifted cohort 53% of features cross PSI 0.25, 85% of basket items are unknown and the rule fire-rate falls to 19% (reference 92%). Normal traffic is not perfectly green: one or two features sit just above PSI 0.10 (amber) with 500 predictions from only 300 customers — sampling noise I report rather than tune away. Each level change raised an alert (7 in total), written to `alert_log`, logged as a JSON `alert` line, posted to `ALERT_WEBHOOK_URL` when set, and — for the deployed service — turned into a GitHub issue by the hourly `monitor.yml`:

| Time (UTC) | Check | Transition | Value | Notified via |
|---|---|---|---|---|
| 13:31:29 | data_quality | green → **amber** | 0.062 | log |
| 13:32:35 | data_quality | amber → **green** | 0.032 | log |
| 13:32:36 | input_drift | green → **amber** | 0.000 | log |
| 13:33:30 | input_drift | amber → **red** | 0.528 | log |
| 13:33:30 | prediction_drift | green → **red** | 2.122 | log |
| 13:33:30 | data_quality | green → **red** | 0.996 | log |
| 13:34:30 | rule_fire_rate | green → **red** | 0.200 | log |

**A design correction found by monitoring.** I first used the pooled training snapshots as the drift reference. Healthy next-quarter traffic was then permanently red: 8% of features had PSI ≥ 0.25 (`tenure_days` 1.15, `std_gap_days` 0.30, `orders_lifetime` 0.26), because the data start in December 2009, so every later snapshot has a longer observable history and lifetime features grow by construction. A monitor that is always red is ignored. The reference is now the calibration snapshot (2,358 rows, demo customers held out), and the training-vs-reference shift is recorded in the reference profile instead of alerted on.

**Delayed ground truth.** The label exists 90 days after a prediction. A weekly backfill joins predictions older than 90 days with the customer's orders and writes `ground_truth`; the dashboard prints the date of the first true accuracy signal (first prediction + 90 days + up to 7 days of job cadence). Replaying the known historical outcomes of the demo customers through this pipeline labelled 2,849 logged predictions (realised PR-AUC 0.768, lapser recall 0.775); these labels are in-sample for the calibrator, so they prove the pipeline, not a new performance estimate.

**Latency.** Server-side `/predict` p95 was 70 ms through Docker Desktop's port proxy on Windows, but 13.5 ms (and `/recommend` 2.2 ms) when called from inside the Docker network: the gap is the proxy delaying POST bodies, not the model. The latency profile by endpoint is D5 work.

## 8. Reproducibility statement (a stranger's commands)

```
git clone https://github.com/OlgaMorgunova/boreal-marche-capstone.git && cd boreal-marche-capstone
docker compose -f docker/compose.yml up --build -d      # API :8000/docs, UI :8501, Postgres, Ollama
python3 scripts/smoke_test.py --api http://127.0.0.1:8000 --ui http://127.0.0.1:8501
pip install "locust>=2.30" && python3 scripts/simulate_traffic.py --seconds 120   # fills the dashboard
```

Without Docker (Windows: `docs/RUN_FROM_ZERO_WINDOWS.md`): `make setup-pinned`, `make api`, `make frontend`, `make simulate`. Rebuilding every artifact from the raw data: `make all` (≈ 15 min). No secret is needed for any of these; optional variables are listed in `.env.example`.

## 9. Limitations and what is pending

Live URLs, the CI/CD run screenshots and the video depend on the AWS bootstrap and the first merge (PENDING above). App Runner's prediction log is per-instance and ephemeral. The deployed assistant has no LLM provider. The Ollama Compose service was not tested end to end. The SageMaker HPO job is still waiting for the spot-instance quota increase (case opened 2026-09-29); D3 §11 is updated when it runs. The fairness gap in the top spend quartile is unresolved (D5).

## 10. Sources and AI-tool use

Course Locust lab (instructor material) for the load-test pattern; AWS documentation for App Runner, ECR and GitHub OIDC federation; Health Canada and Law 25 summaries for the fictional corpus (to be verified against primary sources in D5). AI-assisted coding and drafting tools were used for code, tests, the policy corpus and this text; I reviewed each module and can explain every line. **Where AI-generated output had to be corrected:** the drift reference (pooled training data made healthy traffic red, §7); a test that hid the SKU guardrail by stripping spaces from streamed tokens; the Docker clean-up step that deleted `numpy/_core/tests` and crashed the container at start-up (caught by the container smoke test); and the French detector, which missed accent-free questions.